Policy-as-Detection

for Kubernetes

BSides Sofia · 3 hours

git → ArgoCD → Kyverno → Prometheus → Alert → Ticket

Pre-intro

Who is this for

You work in security, platform, or DevOps.

You have heard of Kubernetes — maybe you even use it.

You do **not** need to be a Kubernetes expert today.

What we are building together

A pipeline that takes **a bad line of YAML**
and turns it into **an actionable security ticket**.

```
Bad YAML  →  Kubernetes  →  Policy engine  →  Metric  →  Alert  →  Ticket
```

Without blocking the deploy.
Without manual audits.
Automatically.

What we will leave with

By the end of this workshop we can all answer:

> "How does a policy violation end up in a ticket?"

And trace the full path:

Git → Argo CD → Kubernetes → Kyverno → Prometheus → Alert → Ticket

The repo

Everything is in Git. Follow along, or clone and run it yourself later.

```
https://github.com/jo114ge/bsides_sofia__detection_policies_2026
```

Structure:

```
apps/          demo application (Kustomize)
policies/      Kyverno ClusterPolicies
gitops/        Argo CD Application objects
observability/ Prometheus rules, Grafana dashboard
deploy/        Helm values for every tool
terraform/     bootstrap (namespaces + ConfigMap)
diagrams/      10 ASCII reference diagrams
slides/        this presentation
```

Block 0

Opening and context

0:00 – 0:10

The problem

Most teams have policies in their Kubernetes clusters.

Those policies do one of two things:

- **Block** the deploy — developer complains, finds a workaround
- **Generate compliance reports** nobody reads until the next audit

In neither case is there **real-time operational visibility**.

What a policy violation actually contains

```
which resource has the problem    →  Deployment/demo-app
in which namespace                →  demo
which rule it violates            →  disallow-latest
owner                             →  platform-team
when it happened                  →  2026-03-21T10:15:32Z
```

This is a **detection signal**.

Not instead of blocking — in addition to it, or before it.

The central question

> "What if instead of using policies only to block things,
> we also used them to detect things?"

> "What if a policy violation did not end in a silent 'denied',
> but in an alert, a dashboard, or a ticket with actionable context?"

What changes

Traditional enforcement	Policy-as-Detection
Blocks or rejects	Measures and makes visible
Output: pass or fail	Output: metric, event, alert
Owned by compliance	Useful for security and ops
Hard to start	Start in Audit, zero impact

Block 1

Kubernetes — the minimum

0:10 — 0:35

Container

A container is a way to package an application with everything it needs to run, so you can execute it the same way on any machine.

```
+-----------------------------------------------+
|  Container                                    |
|                                               |
|  app code + dependencies + runtime            |
|  = runs the same way on any machine           |
+-----------------------------------------------+
```

Pod

A pod is the smallest unit in Kubernetes.
It is where a container runs. Normally one per pod.

```
+----------------------------+
|  Pod                       |
|  +-----------------------+ |
|  |   Container           | |
|  |   image: nginx:1.29.1 | |
|  |   IP: 10.42.0.12      | |
|  +-----------------------+ |
+----------------------------+
```

Kubernetes does **not** manage containers directly — it manages pods.

Deployment

A Deployment tells Kubernetes how many copies of a pod you want.
If a pod dies, Kubernetes recreates it automatically.

```
Deployment  (replicas: 3)

  +-------+   +-------+   +-------+
  |  Pod  |   |  Pod  |   |  Pod  |
  +-------+   +-------+   +--+----+
                                |
                            pod dies
                                ↓
                           +-------+
                           |  Pod  |  ← new
                           +-------+
```

Namespace

A namespace is a logical boundary within the cluster.
Like folders for workloads.

```
+-------------------------------------------------------+
|  Cluster                                              |
|                                                       |
|   +-----------+   +-----------+   +----------+        |
|   | demo      |   | kyverno   |   | argocd   |        |
|   |           |   |           |   |          |        |
|   | demo-app  |   | policies  |   | gitops   |        |
|   +-----------+   +-----------+   +----------+        |
+-------------------------------------------------------+
```

> *Namespaces are **logical** separation, not a security boundary.*

Image and registry

The tag is critical for traceability:

```
nginx:latest    ← which version? unknown. changes silently.
nginx:1.29.1    ← exact version. reproducible. auditable.
```

```
Registry (Docker Hub)          Node
+--------------------+         +------------------+
|  nginx:1.29.1      | ──pull──→  running container |
|  nginx:latest      |         +------------------+
+--------------------+
```

This is why **disallow-latest** exists.

Resource requests

Every container can declare how much CPU and memory it needs.

```yaml
resources:
  requests:
    cpu: 100m        # 0.1 of a CPU core — scheduler uses this
    memory: 128Mi    # guaranteed minimum
  limits:
    cpu: 250m        # hard ceiling
    memory: 256Mi
```

Without requests: the **scheduler is flying blind**.
It may place a pod on a saturated node — causing OOMKills for everyone.

This is why **require-requests** exists.

Admission — the most important concept

When a resource reaches the Kubernetes API server,
before it is accepted, it passes through a **checkpoint**.

```
kubectl apply
     ↓
+--------------------+
|   API server       |
+--------------------+
     ↓
+--------------------+   ← ADMISSION CHECKPOINT
|  Kyverno evaluates |
|  the resource here |
|                    |
|  meets policy?     |
|   yes → continue   |
|   no  → record     |
|          violation |
+--------------------+
     ↓
   etcd  →  resource active in cluster
```

Policy and Violation

Policy — a rule that says how a resource should be configured.

```
# ClusterPolicy: disallow-latest
# rule: if image contains ":latest" → record violation
# mode: Audit (does not block — only records)
```

Violation — when a resource does not satisfy a policy.

```
resource doesn't meet policy
        ↓
  PolicyReport  →  kyverno_policy_results_total{result="fail"}
        ↓
  Prometheus  →  Grafana  →  Alertmanager  →  Ticket
```

The demo app — clean baseline

```yaml
# apps/demo-app/base/deployment.yaml
image: nginxinc/nginx-unprivileged:1.29.1   ← pinned tag  ✓
resources:
  requests:
    cpu: 100m                               ← declared    ✓
    memory: 128Mi
securityContext:
  allowPrivilegeEscalation: false           ← secure      ✓
  runAsNonRoot: true
  runAsUser: 101
```

This manifest is well written. When we introduce a bad config, we will see exactly where it breaks.

Block 2

Policy-as-Detection: the mental model

0:35 – 0:55

Three signals we can get today

Image with `:latest` tag

> *"There is a workload whose provenance you cannot verify."*
> *Supply chain and traceability signal.*

No resource requests

> *"There is a workload that may destabilize its neighbors."*
> *Reliability and cost governance signal.*

Privileged container

> *"There is a workload that can escape its container boundary."*
> *High-severity security signal.*

Audit first — always

> "Before putting up a guardrail, you need to know how many things are already violating that rule. If you block from day one, you break production."

```
Audit mode                  Enforce mode
──────────────              ────────────────────
resource arrives            resource arrives
Kyverno evaluates           Kyverno evaluates
violation? → ALLOW          violation? → REJECT (403)
           → record in              + record in
             PolicyReport               PolicyReport
```

Audit gives us visibility without impact.

Measure the noise. Assign ownership. Then decide on enforcement.

Block 3

End-to-end pipeline architecture

0:55 – 1:10

The complete flow

```
Developer
  ↓  git push
Git repo  (app manifests, policies, observability config)
  ↓  Argo CD syncs
Kubernetes cluster
  ↓  Kyverno evaluates at admission + background
PolicyReport  +  kyverno_policy_results_total{result="fail"}
  ↓  Prometheus scrapes every ~15s
Alert rule: sum(fail) > 0  for 2m
  ↓  FIRING
Alertmanager  →  webhook  →  Slack / Jira / PagerDuty
  ↓  someone acts
Fix committed to Git  →  Argo CD reconciles  →  violation clears
```

Everything is traceable because everything goes through Git.

Role of each tool

Tool	One sentence
Terraform	Bootstrap: creates namespaces, installs tools. Run once.
Argo CD	Watches Git. Every commit syncs to the cluster.
Kyverno	Evaluates resources. Records violations in Audit mode.
Prometheus	Collects `kyverno_policy_results_total`.
Grafana	Visualises the metrics. One panel per policy/namespace.
Alertmanager	Routes the alert to Slack, email, PagerDuty, or webhook.

Full pipeline diagram

```
Developer
   | git push
   v
+-----------------------------------------------------------+
|  Git Repository                                           |
|  apps/demo-app/     policies/kyverno/   observability/   |
+-----------------------------------------------------------+
         | Argo CD syncs          | terraform apply (once)
         v                        v
+-----------------------------------------------------------+
|  Kubernetes Cluster                                       |
|  +------------+   +----------------------------+         |
|  |  Argo CD   |→  |  Admission Checkpoint      |         |
|  +------------+   +----------------------------+         |
|                              ↓                            |
|                   +----------------------------+         |
|                   |   Kyverno  (Audit mode)    |         |
|                   +----------------------------+         |
|                     ↙ allow          ↘ record            |
|              +----------+      +-------------+           |
|              | demo-app |      | PolicyReport|           |
|              +----------+      +-------------+           |
+-----------------------------------------------------------+
                                     | kyverno_policy_results_total
                                     v
+-----------------------------------------------------------+
|  Observability Stack                                      |
|  Prometheus → Grafana dashboard                           |
|  Prometheus → Alertmanager → webhook → Ticket/Slack       |
+-----------------------------------------------------------+
```

Block 4

Terraform: declarative bootstrap

1:10 – 1:25

What Terraform does here

```
# terraform/main.tf
resource "kubernetes_namespace" "demo" {
  metadata {
    name = "demo"
    labels = {
      owner = "platform-team"   ← violation tickets use this
      env   = "workshop"
    }
  }
}

resource "kubernetes_config_map" "workshop_metadata" {
  # cluster name, team, mode
  # enriches alerts and tickets
}
```

> "Terraform is not the star of this workshop.
> It is the layer that makes the platform reproducible.
> Run it once. GitOps handles day-to-day operations."

Helm and Kustomize — two different layers

	Helm	Kustomize
Used for	Third-party tools	Your own app
How	`helm install` + values file	`kubectl apply -k` + overlays
In workshop	ArgoCD, Kyverno, Prometheus	demo-app base + patches

```
# Helm installs the tools (we don't touch the YAMLs)
helm upgrade --install argocd argo/argo-cd \
  --values deploy/helm-values/argocd-values.yaml

# Kustomize manages the app (we do touch these YAMLs)
kubectl apply -k apps/demo-app/overlays/workshop
```

Block 5

Argo CD: GitOps baseline

1:25 – 1:50

The GitOps reconciliation loop

```
DESIRED STATE (Git)                    ACTUAL STATE (cluster)
+-------------------------+            +-------------------------+
|  apps/demo-app/         |            |  Deployment/demo-app    |
|    image: nginx:1.29.1  |            |    image: nginx:1.29.1  |
|    replicas: 1          |            |    replicas: 1          |
+-------------------------+            +-------------------------+
              ↘                                      ↗
                    +------------------+
                    |    Argo CD       |
                    |  detects diff?   |
                    |                  |
                    |  NO  → Synced    |
                    |  YES → apply diff|
                    +------------------+
```

`selfHeal: true` — if someone changes the cluster directly, Argo CD reverts it.

`prune: true` — resources removed from Git are deleted from the cluster.

The Application object

```yaml
# gitops/argocd/demo-app.yaml
spec:
  source:
    repoURL: https://github.com/jo114ge/bsides_sofia__detection_policies_2026
    path: apps/demo-app/overlays/workshop
    targetRevision: HEAD
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

> "Argo CD does not manage containers. It does not build images.
> It is the CD part only: it watches Git and applies changes."

Exercise 1 — Safe change

```
# Change replicas: 1 → replicas: 2 in apps/demo-app/base/deployment.yaml
git add apps/demo-app/base/deployment.yaml
git commit -m "scale demo-app to 2 replicas"
git push
```

```
kubectl --context k3d-workshop -n demo rollout status deployment/demo-app
kubectl --context k3d-workshop -n demo get pods
```

We changed a number in a YAML file. Pushed it to Git.
Argo CD detected the drift and applied it.
Nobody ran `kubectl apply` manually.

This is the **same flow** we will use to introduce the violation and the fix.

Block 6

Main demo: from violation to detection

1:50 – 2:25

The four states

```
State 0  →  clean baseline
State 1  →  introduce bad configuration
State 2  →  Kyverno detects the violation
State 3  →  violation becomes observable metric
State 4  →  the ticket
```

State 0 — Clean baseline

```
kubectl --context k3d-workshop -n demo get pods
kubectl --context k3d-workshop get policyreport -A
```

Expected: app running, no violations, Grafana shows 0.

> "This is the starting point.
> Now we are going to intentionally introduce a bad configuration."

State 1 — Introduce the bad config

```
cat apps/demo-app/overlays/workshop/bad-image-patch.yaml
```

```yaml
# This patch overwrites ONLY the image field
spec:
  template:
    spec:
      containers:
        - name: web
          image: nginxinc/nginx-unprivileged:latest  ← violates disallow-latest
```

Activate it in `kustomization.yaml`:

```yaml
patches:
  - path: bad-image-patch.yaml
    target: { kind: Deployment, name: demo-app }
```

```
kubectl --context k3d-workshop apply -k apps/demo-app/overlays/workshop
```

State 2 — Kyverno detects the violation

```
kubectl --context k3d-workshop get policyreport -A -o json | \
  jq '.items[].results[] | select(.result=="fail")'
```

```
{
  "message":  "Images must not use the latest tag.",
  "policy":   "disallow-latest",
  "result":   "fail",
  "resources": [{
    "kind":      "Deployment",
    "name":      "demo-app",
    "namespace": "demo"
  }]
}
```

> "The app is still running. Audit mode did not block it.
> But the violation is recorded. We have the policy name, resource, namespace."

How Kyverno detected it

```
Developer / CI                API Server             Kyverno Webhook
     |                            |                          |
     |--- kubectl apply --------->|                          |
     |                            |--- AdmissionReview ----->|
     |                            |                          |
     |                            |         Audit mode       |
     |                            |              ↓           |
     |                            |        violation?        |
     |                            |        YES → Allow       |
     |                            |              + record in PolicyReport
     |<-- resource accepted <-----|
```

Also: **background scan** — Kyverno re-evaluates all existing resources periodically.
That is why it detects the violation even after the resource is already deployed.

State 3 — The metric

```
cat observability/prometheus/rules.yaml
```

```
alert: KyvernoPolicyViolationDetected
expr: sum by (policy, namespace) (kyverno_policy_results_total{result="fail"}) > 0
for: 2m      ← wait before firing (avoids noise from transient restarts)
labels:
  severity: warning
  team: platform
```

Open Grafana → http://localhost:3000

The violations panel should show **1** for `disallow-latest` in namespace `demo`.

> If it has not moved yet: Prometheus has a scrape interval. Wait 30 seconds.

Prometheus alert states

```
expr FALSE
     ↓ expression becomes TRUE
  PENDING ──────── 2 minutes ──────────→ FIRING
     ↓                                       ↓
  expr becomes FALSE                 expr becomes FALSE
  (before 2m)                        after firing
     ↓                                       ↓
  (silent, no alert sent)              RESOLVED
                                       (Alertmanager notified)
```

Timing from violation to notification:

Event	Delay
PolicyReport updated	~0s (immediate)
Metric visible in Prometheus	~15s
Alert enters PENDING	~15s
Alert enters FIRING	~2m 15s

State 4 — The ticket

```
{
  "summary":     "Policy violation: unpinned image in demo-app",
  "severity":    "medium",
  "owner":       "platform-team",
  "namespace":   "demo",
  "resource":    "Deployment/demo-app",
  "policy":      "disallow-latest",
  "evidence":    "Container image uses nginx:latest",
  "remediation": "Pin the image to a versioned tag and merge via Git."
}
```

Without context: "there is a security problem in Kubernetes"
With context: "demo-app in namespace demo uses nginx:latest, owner is platform-team, fix is to pin the image and merge via Git"

Same ticket. One is noise. The other is an actionable signal.

Block 7

Dashboards and alerts

2:25 – 2:45

What a useful dashboard answers

> "A useful violations dashboard answers three questions:
> What is broken? Where is it broken? How long has it been like this?"

Panels with value:

- Active violations by policy — which rules are being violated most
- Violations by namespace — which team has the most config debt
- Trend over time — does the number go up or down after a remediation sprint

Panels without value:

- Historical totals without time filter
- Metrics without breakdown by owner or namespace
- Alerts that always fire and nobody looks at

What a good alert answers

> "A good alert answers one question:
> what should someone do right now?
> If the alert does not have that answer, it is noise."

```yaml
annotations:
  summary:     "Kyverno policy violation detected"
  description: "Policy {{ $labels.policy }} has active failures
                in namespace {{ $labels.namespace }}"
```

The description names the policy and the namespace.
Whoever receives it knows in two seconds whether they need to act.

Alert hygiene

> "Do not alert on everything.
> If you configure 50 policies and all of them generate alerts,
> nobody will look at any of them."

Recommended progression:

```
Week 1–2   →  Audit mode, dashboards only. Measure the noise.
Week 3–4   →  Add alerts for 2–3 high-impact policies.
Month 2    →  Tune severity, assign ownership.
Month 3+   →  Consider Enforce for the most critical ones.
```

Block 8

Tickets and workflow

2:45 – 2:55

Mandatory fields for a useful ticket

Field	Why mandatory
`policy`	team knows which rule to look at
`resource`	without this, you have to investigate
`namespace`	determines the owner team
`owner`	without owner, ticket dies in the queue
`evidence`	concrete proof, no ambiguity
`remediation`	what to do — must be actionable

> "A ticket nobody is going to resolve is more harmful than no ticket.
> It takes up space, lowers confidence, and gets ignored like alert noise."

Block 9

Fix via GitOps and closing

2:55 – 3:00

The fix

The same mechanism that introduced the problem brings the fix.

```
# Remove the patches: section from kustomization.yaml
kubectl --context k3d-workshop apply -k apps/demo-app/overlays/workshop
kubectl --context k3d-workshop -n demo rollout status deployment/demo-app
```

```
kubectl --context k3d-workshop get policyreport -n demo -o json | \
  jq '.items[].results[] | select(.result=="fail")'
# Expected output: empty
```

> "The violation has disappeared.
> The metric will drop to zero in the next scrape cycle.
> The alert resolves itself. The ticket can be closed."

The complete flow — recap

```
1. Git      →  source of truth for app, policy, and observability
2. Argo CD  →  syncs Git to the cluster
3. Kyverno  →  evaluates in Audit mode, records violation in PolicyReport
4. Prometheus  →  collects the violation metric
5. Grafana / Alertmanager  →  visibility and alerting
6. Ticket   →  actionable signal with context
7. Fix via Git  →  Argo CD reconciles, signal clears
```

Top 5 for Monday

1. Pick a **simple policy** and put it in `Audit` mode
2. Make at least one violation **visible in a dashboard**
3. Create an **actionable alert** from that policy
4. Define **owner and severity** for that type of violation
5. Manage that change **from Git** instead of doing it manually

> "A policy nobody sees is compliance.
> A policy that creates action is detection."

Block 10

Sending violations to the SIEM

Optional · security / SOC audience

The format gap

```
Kyverno produces:                SIEM expects:
─────────────────                ─────────────────────────
PolicyReport (K8s object)        CEF / ECS / LEEF / JSON
Prometheus metric                Syslog, HTTP ingest, Kafka
Kubernetes event                 Normalized fields
```

Three ways to close the gap:

Path	Latency	Extra components
Alertmanager webhook	~2–3 min	none (already wired)
Log shipper (Fluentd/Vector)	~seconds	DaemonSet per node
policy-reporter	~seconds	1 Deployment

Path 1 — Alertmanager webhook

Already in the repo. Change the URL:

```yaml
# observability/alertmanager/receivers.yaml
receivers:
  - name: siem-webhook
    webhook_configs:
      - url: https://your-siem.company.com/api/ingest
        send_resolved: true
```

Pros: zero extra components, works today.

Cons: 2-min delay, one alert per policy/namespace (no per-resource detail).

Path 3 — policy-reporter (recommended)

```
helm install policy-reporter policy-reporter/policy-reporter \
  --set target.elasticsearch.enabled=true \
  --set target.elasticsearch.host=https://your-elastic:9200
```

Normalized event output:

```
{
  "policy":    "disallow-latest",
  "result":    "fail",
  "severity":  "medium",
  "category":  "Software Supply Chain",
  "resource":  { "kind": "Deployment", "name": "demo-app", "namespace": "demo" },
  "message":   "Images must not use the latest tag.",
  "cluster":   "k3d-workshop"
}
```

Three SIEM rule types

Threshold — any critical violation fires a ticket:

```
index=kubernetes_policy result=fail policy=disallow-privileged
| stats count by namespace, resource | where count > 0
```

Correlation — 3+ policies violated in same namespace in 10 minutes:

```
index=kubernetes_policy result=fail earliest=-10m
| stats dc(policy) as distinct_policies by namespace
| where distinct_policies >= 3
```

Anomaly — spike above baseline (start here after 1 month, not day 1).

Enrichment to add before ingestion

Field	Source	Why
`owner_team`	namespace label `owner`	who to page
`env`	namespace label `env`	prod vs staging priority
`cluster_name`	ConfigMap `workshop-metadata`	multi-cluster
`git_commit`	Deployment annotation	correlate with the change
`image_digest`	pod spec	exact image, not just tag

Block 11

Extended scenarios

Optional · security audience

Six policies, same pipeline

#	Policy	Severity	Signal
1	`disallow-latest`	medium	Supply chain / traceability
2	`require-requests`	medium	Reliability / cost
3	`disallow-privileged`	high	Container escape risk
4	`disallow-run-as-root`	high	Privilege escalation
5	`restrict-image-registries`	high	Untrusted code
6	`disallow-host-namespaces`	critical	Sandbox broken

All patches are in `apps/demo-app/overlays/workshop/`. Uncomment to activate.

Scenario 3 — disallow-privileged

```yaml
# bad-privileged-patch.yaml
securityContext:
  privileged: true
  allowPrivilegeEscalation: true
```

> "A privileged container has nearly full access to the host node.
> If an attacker exploits a CVE inside, they can break out,
> read Kubernetes certificates at /etc/kubernetes/pki,
> and take over the entire cluster."

Severity: HIGH — investigate immediately.

Scenario 4 — disallow-run-as-root

```yaml
# bad-run-as-root-patch.yaml
securityContext:
  runAsNonRoot: false
  runAsUser: 0
```

> "Most container breakout CVEs require the process to be root.
> Non-root containers are significantly harder to exploit.
> Every container running as UID 0 needs re-examining."

Severity: HIGH

Scenario 5 — restrict-image-registries

```
# bad-image-registry-patch.yaml
image: docker.io/library/nginx:1.29.1  ← not from internal registry
```

> "Docker Hub has thousands of typosquatted images.
> Your internal registry runs vulnerability scans. Public registries do not.
> This is the same attack surface as dependency confusion — for containers."

Severity: HIGH

Scenario 6 — disallow-host-namespaces

```yaml
# bad-host-namespaces-patch.yaml
spec:
  hostNetwork: true
  hostPID: true    ← can read /proc/<pid>/environ of any process on the node
  hostIPC: true
```

> "hostPID lets the container see all processes on the node.
> It can read environment variables of the kubelet —
> which may contain cloud provider credentials passed at boot."

Severity: CRITICAL — escalate immediately.

Closing

The pattern is always the same

```
Bad config committed to Git
         ↓
Argo CD syncs to cluster
         ↓
Kyverno evaluates (Audit mode)
         ↓
PolicyReport  +  Prometheus metric
         ↓
Grafana panel  +  Alert rule
         ↓
Alertmanager  →  Slack / Jira / SIEM / PagerDuty
         ↓
Someone acts  →  fix via Git  →  signal clears
```

The two policies in the workshop are medium severity.
The same pipeline works for CRITICAL.

Reference diagrams — take these home

```
cat diagrams/1-k3d-cluster-ascii.txt        # cluster structure
cat diagrams/2-kubernetes-concepts-ascii.txt # K8s vocabulary
cat diagrams/3-apps-and-policies-ascii.txt   # 6 policy scenarios
cat diagrams/4-architecture-ascii.txt        # full pipeline
cat diagrams/5-argocd-flow-ascii.txt         # GitOps loop
cat diagrams/6-kyverno-how-it-works-ascii.txt# admission + background
cat diagrams/7-prometheus-alert-flow-ascii.txt# PromQL + alert states
cat diagrams/8-grafana-how-it-works-ascii.txt # panels + timing
cat diagrams/9-helm-kustomize-ascii.txt      # Helm vs Kustomize
cat diagrams/10-siem-integration-ascii.txt   # 3 paths + rule types
```

> "A policy nobody sees is compliance.
> A policy that creates action is detection."

Thank you

Repo: `github.com/jo114ge/bsides_sofia__detection_policies_2026`

Presenter guide: `docs/presenter-guide.md`

Diagrams: `diagrams/1-*.txt` through `diagrams/10-*.txt`

BSides Sofia · 2026